
JAVA EXCEPTION HANDLING

try-catch-finally • throw • throws • Custom Exceptions • Framework-Level Handling

13+ Years Senior SDET Interview Guide

What this guide covers

Core syntax, exception hierarchy, checked vs unchecked exceptions, multiple catch blocks, finally, throw/throws, propagation, chaining, try-with-resources, custom exceptions, Selenium/TestNG handling, REST Assured handling, and senior framework design.

Interview mindset

Do not say “I catch exceptions so the test does not fail.” A mature framework adds context, logs useful diagnostics, preserves the cause, performs cleanup, and propagates the failure correctly.

1. Core try-catch Fundamentals

Q1. What is exception handling in Java?

Exception handling allows a program to respond to abnormal runtime conditions in a controlled way instead of allowing an unexpected failure to terminate the intended execution path.

Code example

```
public class ExceptionExample {  
    public static void main(String[] args) {  
        try {  
            // Risky code that may throw an exception.  
            int result = 10 / 0;  
  
            System.out.println(result);  
        } catch (ArithmeticException e) {  
            // Handle the specific exception.  
            System.out.println("Cannot divide by zero.");  
        }  
    }  
}
```

Q2. What is a try block?

A try block contains code that may throw an exception. It must be followed by at least one catch block or a finally block.

Code example

```
try {  
    // Code that may fail.  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    // Handle the failure.  
}
```

Q3. What is a catch block?

A catch block handles an exception thrown from its associated try block. Prefer a specific exception type when you know what failure you are handling.

Code example

```
try {  
    // Risky operation.  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println(  
        "Arithmetic exception occurred."  
    );  
}
```

Q4. What is finally?

finally is normally used for cleanup that should happen regardless of whether the try block succeeds or a matching exception is caught.

Code example

```
try {  
    // Main operation.  
} catch (Exception e) {  
    // Error handling.  
} finally {  
    // Cleanup.  
    // Example: close a resource.  
}
```

Q5. Can try exist without catch?

Yes. A try block can be followed by finally without catch.

Code example

```
try {  
    System.out.println("Execute operation");  
} finally {  
    // Cleanup is performed.  
    System.out.println("Cleanup");  
}
```

Q6. Can try exist without catch and finally?

No. A try block must have at least one catch block or a finally block.

Code example

```
// Invalid Java:  
  
try {  
    System.out.println("Hello");  
}  
  
// Compilation error because there is  
// neither catch nor finally.
```

2. Multiple Catch Blocks and Exception Hierarchy

Q7. Can we have multiple catch blocks?

Yes. Java checks catch blocks from top to bottom and executes the first compatible handler.

Code example

```
try {  
    int[] numbers = {1, 2, 3};  
    System.out.println(numbers[5]);  
} catch (ArithmeticException e) {  
    System.out.println("Arithmetic problem");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Invalid array index");  
} catch (Exception e) {  
    System.out.println("Generic exception");  
}
```

Q8. What is the correct order of multiple catch blocks?

A child exception must appear before its parent. Otherwise the parent catch makes the child catch unreachable.

Code example

```
try {  
    // Risky code.  
} catch (ArithmeticException e) {  
    // Specific exception first.  
} catch (RuntimeException e) {  
    // Parent of ArithmeticException.  
} catch (Exception e) {  
    // Broader parent last.  
}
```

Q9. Explain the exception hierarchy.

Throwable is the root. Its major branches are Error and Exception. RuntimeException is under Exception and represents unchecked exceptions.

Code example

```
Throwable
|
+-- Error
|   +-- OutOfMemoryError
|   +-- StackOverflowError
|
+-- Exception
    |
    +-- RuntimeException
    |   +-- NullPointerException
    |   +-- ArithmeticException
    |   +-- IllegalArgumentException
    |
    +-- IOException
    +-- SQLException
    +-- InterruptedException
```

Q10. What is the difference between Error and Exception?

Exceptions are conditions applications may often handle or propagate. Errors generally indicate serious JVM or system-level problems. Do not use broad Throwable/ Error catching as a default framework strategy.

Code example

```
try {
    // Application operation.
} catch (Exception e) {
    // Handle or propagate expected application failures.
}

// Avoid using catch(Throwable) as a general
// mechanism to hide all failures.
```

3. Checked vs Unchecked Exceptions

Q11. What is a checked exception?

A checked exception is checked by the compiler. Code generally must handle it or declare it in the method signature. Examples include `IOException` and `SQLException`.

Checked exception example

```
public void readFile() throws IOException {  
    // FileReader may produce a checked IOException.  
    FileReader reader =  
        new FileReader("data.txt");  
}
```

Q12. What is an unchecked exception?

Unchecked exceptions are `RuntimeException` subclasses. The compiler does not require them to be explicitly caught or declared.

Unchecked exception example

```
public void validateAge(int age) {  
    if (age < 18) {  
        // RuntimeException subclass.  
        throw new IllegalArgumentException(  
            "Age must be at least 18"  
        );  
    }  
}
```

Interview tip

Do not say checked exceptions are always recoverable and unchecked exceptions are always programming bugs. Explain the language distinction, then discuss the application/framework handling strategy.

Q13. What is exception propagation?

If a method does not handle an exception, it can propagate up the call stack to its caller until a compatible handler is found. If no handler is found, the exception reaches the top-level execution boundary.

Exception propagation

```
public static void method1() {  
    // Calls method2().  
    method2();  
}  
  
public static void method2() {  
    // Calls method3().  
    method3();  
}  
  
public static void method3() {  
    // Exception originates here.  
    int result = 10 / 0;  
}  
  
// Propagation concept:  
//  
// method1()  
//     -> method2()  
//         -> method3()  
//             -> ArithmeticException
```

4. throw, throws and Exception Chaining

Q14. Difference between throw and throws?

throw explicitly creates/throws an exception. throws declares that a method may propagate one or more exceptions.

Code example

```
public void validate(int age)
    throws IOException {

    if (age < 18) {

        // Explicitly throw an unchecked exception.
        throw new IllegalArgumentException(
            "Invalid age"
        );
    }

    // A checked exception could also be thrown
    // if declared or handled.
}
```

Q15. Can throw and throws be used together?

Yes. A method can declare a checked exception with throws and explicitly throw an exception using throw.

Code example

```
public void process()
    throws IOException {

    if (someCondition()) {

        throw new IOException(
            "Processing failed"
        );
    }
}
```

Q16. What is exception chaining?

Exception chaining preserves the original cause when a lower-level exception is wrapped in a meaningful higher-level exception.

Code example

```
try {

    // Database/API operation.

} catch (SQLException e) {

    // Add domain-level context while preserving
    // the original SQLException as the cause.
    throw new TestDataException(
        "Unable to retrieve test data",
        e
    );
}
```

Q17. Why preserve the original cause?

The cause contains the low-level diagnostic information. Preserving it makes the complete failure chain visible during troubleshooting.

Code example

```
throw new TestDataException(  
    "Unable to retrieve test data",  
    originalException  
);  
  
// Conceptually:  
//  
// TestDataException  
//     |  
//     +--> SQLException  
//     |  
//     +--> original database error
```

5. finally — Advanced Interview Questions

Q18. Does finally execute when try has return?

Normally yes. The finally block runs before the method completes its return.

Code example

```
public static int test() {  
    try {  
        return 10;  
    } finally {  
        // Executes before the return completes.  
        System.out.println("Finally");  
    }  
}
```

Q19. What happens if both try and finally return?

The return from finally can override the earlier return. This is why returning from finally is strongly discouraged.

Code example

```
public static int test() {  
    try {  
        return 10;  
    } finally {  
        // Overrides the try return.  
        return 20;  
    }  
}  
  
// Result: 20
```

Q20. What happens if finally throws an exception?

The exception from finally can propagate and can mask an exception that was already being propagated from try or catch. Cleanup code should therefore be simple and carefully designed.

Code example

```
try {  
    throw new RuntimeException(  
        "Original exception"  
    );  
} finally {  
    // Dangerous: this can mask the original failure.  
    throw new RuntimeException(  
        "Finally exception"  
    );  
}
```

Q21. Can finally be skipped?

Normally finally executes, but abrupt JVM/process termination can prevent normal execution. For example, `System.exit()` terminates the JVM.

Code example

```
try {  
    System.out.println("Try");  
  
    // Abruptly terminates the JVM.  
    System.exit(0);  
  
} finally {  
    // Normally not reached after System.exit().  
    System.out.println("Finally");  
}
```

6. Modern Java Exception Handling

Q22. What is try-with-resources?

Try-with-resources automatically closes resources implementing `AutoCloseable`, reducing manual cleanup code and making resource handling safer.

Try-with-resources

```
try (
    FileReader reader =
        new FileReader("data.txt");

    BufferedReader bufferedReader =
        new BufferedReader(reader)
) {

    // Use the resource.
    String line =
        bufferedReader.readLine();

    System.out.println(line);
} catch (IOException e) {

    // Handle the I/O failure.
    System.out.println(e.getMessage());
}

// Resources are closed automatically.
```

Q23. Can multiple resources be declared?

Yes. Resources can be declared in the try header. They are closed automatically in the reverse order of resource acquisition.

Multiple resources

```
try (
    FileInputStream input =
        new FileInputStream("input.txt");

    BufferedInputStream buffered =
        new BufferedInputStream(input)
) {

    // Process data.
} catch (IOException e) {

    // Handle I/O error.
}
```

Q24. What is multi-catch?

Multi-catch lets one catch block handle multiple unrelated exception types when the handling strategy is the same.

Multi-catch

```
try {  
    // Operation.  
} catch (IOException | SQLException e) {  
    // Same handling strategy for both types.  
    logger.error(  
        "Operation failed",  
        e  
    );  
}
```

7. Custom Exceptions and Framework Design

Q25. What is a custom exception?

A custom exception is an application/framework-specific exception class that gives failures a meaningful domain name.

Custom framework exception

```
public class TestDataException
    extends RuntimeException {

    public TestDataException(String message) {

        super(message);
    }

    public TestDataException(
        String message,
        Throwable cause) {

        // Preserve the original cause.
        super(message, cause);
    }
}
```

Q26. Why create custom exceptions in an SDET framework?

They allow the framework to communicate meaningful failure categories such as ConfigurationException, DriverInitializationException, TestDataException or ApiClientException instead of exposing low-level implementation details everywhere.

Framework-level exception wrapping

```
try {

    // Read framework configuration.

} catch (IOException e) {

    // Convert low-level failure into a meaningful
    // framework-level exception.
    throw new ConfigurationException(
        "Unable to load environment configuration",
        e
    );
}
```

Senior design rule

Catch an exception where you can add useful context or take a meaningful action. Do not catch exceptions merely to stop a test from failing.

8. Selenium / TestNG Exception Handling

Q27. How should Selenium exceptions be handled?

Catch a specific Selenium exception when you can add useful context, capture diagnostics if appropriate, then propagate the failure so TestNG records the test correctly.

Selenium utility exception handling

```
public void click(By locator) {  
    try {  
        // Perform the browser action.  
        driver.findElement(locator).click();  
    } catch (NoSuchElementException e) {  
        // Add locator context to the log.  
        logger.error(  
            "Element not found: " + locator,  
            e  
        );  
        // Re-throw so the test fails correctly.  
        throw e;  
    }  
}
```

Q28. Why is this bad?

Exception swallowing — avoid

```
try {  
    driver.findElement(locator).click();  
} catch (Exception e) {  
    // BAD: exception is swallowed.  
    // The test may continue and report incorrectly.  
}
```

Q29. How should WebDriver cleanup be handled?

Use framework teardown such as TestNG @AfterMethod/@AfterClass according to the driver lifecycle. Always make cleanup execute even when the test fails. With ThreadLocal, remove the thread association after quitting the driver.

TestNG + ThreadLocal cleanup

```
@AfterMethod(alwaysRun = true)
public void tearDown() {

    try {

        // Get the driver for this test/thread.
        WebDriver driver =
            DriverManager.getDriver();

        if (driver != null) {

            // End the browser session.
            driver.quit();
        }

    } finally {

        // Remove ThreadLocal state.
        DriverManager.removeDriver();
    }
}
```

9. REST Assured / API Framework Exception Handling

Q30. How would you handle exceptions in a REST Assured API client?

Keep API execution and validation responsibilities clear. Catch technical client-side failures where you can add request context, log the endpoint or test identifier, preserve the original cause, and rethrow a meaningful API exception. Do not hide HTTP assertion failures or return null to conceal an error.

REST Assured framework example

```
public Response getUser(String userId) {  
    try {  
        return given()  
            .pathParam("id", userId)  
            .when()  
            .get("/users/{id}");  
    } catch (Exception e) {  
        logger.error(  
            "GET /users failed for userId: " + userId,  
            e  
        );  
        // Preserve the original cause.  
        throw new ApiClientException(  
            "Unable to execute GET user API",  
            e  
        );  
    }  
}
```

Q31. Why is catch(Exception) sometimes acceptable at a framework boundary but not everywhere?

At a controlled framework boundary, a broad catch can be useful when the framework needs to add standard logging/diagnostics and translate technical failures into a common exception. It should not be used deep inside business actions to indiscriminately hide specific failures.

Controlled exception boundary

```
// Framework boundary:  
// acceptable when adding standard context.  
  
try {  
    // Execute framework operation.  
} catch (Exception e) {  
    // Log standard framework context.  
    logger.error(  
        "Framework operation failed",  
        e  
    );  
    // Preserve cause and propagate.  
    throw new FrameworkException(  
        "Framework operation failed",  
        e  
    );  
}
```

10. Senior SDET Interview Scenarios

Q32. How do you prevent false-positive automation tests?

Never swallow an exception. If a business action fails, propagate the failure so the test framework records the test as failed. Add diagnostics before propagation.

Q33. How do you decide what to catch?

Catch the most specific exception you can meaningfully handle. If you cannot recover or add useful context, allow the exception to propagate.

Q34. Should every method have try-catch?

No. Excessive try-catch creates noisy code and can hide failures. Place exception handling at appropriate architectural boundaries.

Q35. What is your framework exception strategy?

Use specific exceptions at low-level utilities, meaningful custom exceptions at framework boundaries, centralised logging/reporting, deterministic cleanup, and exception propagation that preserves the original cause.

Q36. What information should be logged?

Log useful context such as operation name, test name, locator or endpoint, relevant identifiers, environment and the exception stack trace. Avoid logging secrets such as passwords or access tokens.

Senior SDET answer

"I do not use try-catch simply to prevent test failure. I catch where I can add meaningful context, log the relevant test/UI/API information, capture diagnostics where appropriate, preserve the original cause, and rethrow or wrap the exception. Cleanup belongs in finally or test teardown. This keeps failures visible, actionable and correctly reported."

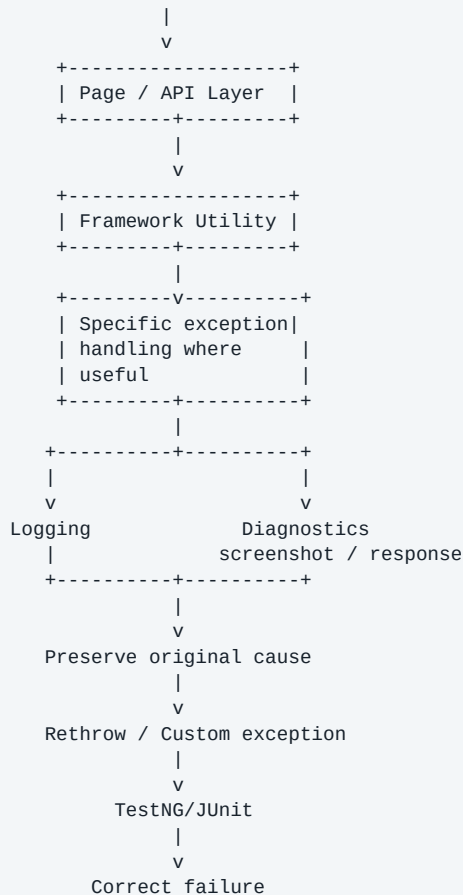
11. Interview Rapid Revision — 25 Questions

- Q37. Can try exist without catch? — Yes, when followed by finally.
- Q38. Can try exist without catch and finally? — No.
- Q39. Can one try have multiple catch blocks? — Yes.
- Q40. Which catch should come first? — Specific child exception before parent.
- Q41. What does finally do? — Normally performs cleanup regardless of success/caught failure.
- Q42. What is exception propagation? — An unhandled exception moves up the call stack.
- Q43. What is throw? — Explicitly throws an exception object.
- Q44. What is throws? — Declares possible propagated exceptions in a method signature.
- Q45. What is exception chaining? — Wrapping while preserving the original cause.
- Q46. What is a checked exception? — Compiler-checked exception requiring handling or declaration.
- Q47. What is an unchecked exception? — RuntimeException subclass not required to be declared/caught.
- Q48. What is multi-catch? — One catch handling multiple exception types.
- Q49. What is try-with-resources? — Automatic resource cleanup for AutoCloseable resources.
- Q50. What is a custom exception? — A framework/application-specific exception type.
- Q51. Why avoid empty catch blocks? — They swallow failures and hide root causes.
- Q52. Why avoid return in finally? — It can override returns and mask exceptions.
- Q53. Can finally throw? — Yes, and it can mask another failure.
- Q54. Can finally be skipped? — Yes under abrupt JVM/process termination such as System.exit().
- Q55. Why rethrow Selenium exceptions? — To keep the test result accurate.
- Q56. Why use ThreadLocal cleanup? — To remove per-thread state when worker threads are reused.
- Q57. Should we catch Throwable? — Not as a general-purpose error hiding mechanism.
- Q58. What is exception wrapping? — Converting a lower-level exception into a higher-level one while preserving cause.
- Q59. What is exception swallowing? — Catching and doing nothing or otherwise hiding the failure.
- Q60. What is the senior-level goal? — Actionable diagnostics, correct failure propagation and deterministic cleanup.

12. Final Framework Exception-Handling Blueprint

Senior SDET exception-handling architecture

TEST



CLEANUP PATH:

```
finally / @AfterMethod(alwaysRun = true)
|
+--> quit WebDriver
|
+--> close resources
|
+--> ThreadLocal.remove()
```

■ Final answer to memorise

"My exception-handling strategy is to keep failures visible rather than hide them. I catch exceptions at the appropriate framework boundary, add useful context and diagnostics, preserve the original cause, and rethrow or wrap them in meaningful framework exceptions. I use finally or TestNG teardown for deterministic resource cleanup. In parallel Selenium execution I isolate WebDriver instances per thread and remove ThreadLocal state during teardown. This gives accurate test results and actionable failures."